

REGRESSION METRICS

Metrics For Model Evaluation

Now we have trained our models and they are ready to make predictions over test dataset. To make sure the model is predicting meaningful values not random output we need some metrics to evaluate the output. This is where performance metrics come into picture.

Let's implement some of the most widely used evaluation metrics for regression problems from scratch.

Performance Metrics Implementation

Mean Squared Error (MSE)

It is estimated as average squared differences of predicted values and the actual values. For an ideal model mse would be equal to 0. Lower mse depicts more accurate predictions. It is expressed in squared units of the target variable

Mathematically it is expressed as :

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_{\text{pred},i} - y_{\text{true},i})^2$$

Note that the above expression is differential and hence can we calculate gradient for optimizing purposes.

```
"""
    Calculate Mean Squared Error (MSE) between true and predicted values.

    Parameters:
        -- y_true (numpy array or list): True values
        -- y_pred (numpy array or list): Predicted values

    Returns:
        -- mse (float): Mean Squared Error
    """

def mean_squared_error(y_true, y_pred):

    #convert then into numpy array if not already
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    #squared the differences
    squared_diff = (y_true - y_pred)**2
    #find the avg of squared_diff
    mse = np.mean(squared_diff)
    return mse
```

Bounds of mean-squared error

$[0, \infty)$, with 0 the best.

Cautions

- **Sensitive to outliers:** MSE is sensitive to outliers because if a predicted value (y_{pred_i}) is an outlier, the squared difference with the corresponding true value (y_{true_i}) will be larger, potentially resulting in an inflated MSE.
- **Imbalanced errors:** MSE treats all errors equally, regardless of their magnitude or direction. This means that overestimation and underestimation errors are weighted equally, even though they may have different implications or costs in practice.
- **Not robust to non-Gaussian errors:** MSE assumes that the errors follow a Gaussian (normal) distribution, which may not always be the case in real-world scenarios. If the errors are not normally distributed, MSE may not accurately reflect the model's performance.
- **Lack of sensitivity to small errors:** MSE may not capture small errors well, as it squares the differences between predicted and true values. This can lead to a model with good MSE but poor performance in capturing small errors, which may be important in certain applications.

Root Mean Squared Error(RMSE)

This is root of mean squared error we talked about. The good thing about this metric is like the unit is in sync with target variable and hence can be used for comparing multiple models at once.

Since RMSE is expressed in the same units as the dependent variable, making it easy to interpret in the context of the original data

The expression of RMSE is differentiable just like we have for MSE.

```
"""
    Calculate Root Mean Squared Error (RMSE) between true and predicted values.

    Parameters:
        -- y_true (numpy array or list): True values
        -- y_pred (numpy array or list): Predicted values

    Returns:
        -- mse (float): Mean Squared Error
    """

def root_mean_squared_error(y_true, y_pred):

    #convert then into numpy array if not already
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    #squared the differences
    squared_diff = (y_true - y_pred)**2
    #find the avg of squared_diff
    mse = np.mean(squared_diff)
    return np.sqrt(mse)
```

Cautions

- Not robust to outliers
- Always positive so not useful when negative errors make sense.
- Bias towards large values
- Can be affected by sample size

Mean Absolute Error (MAE)

It is the average of absolute difference of predicted and actual values. It is more robust to outliers unlike MSE and RMSE.

Mathematically, It can be expressed as:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_{\text{true}_i} - y_{\text{pred}_i}|$$

Note that it is not differential at zero because of the mod $|\cdot|$ function.

```
"""
    Calculate Mean Absolute Error (MAE) between true and predicted values.

    Parameters:
        -- y_true (numpy array or list): True values
        -- y_pred (numpy array or list): Predicted values

    Returns:
        -- mae (float): Mean Absolute Error
    """

def mean_absolute_error(y_true, y_pred):

    #convert then into numpy array if not already
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    #squared the differences
    absolute_diff = np.absolute(y_true - y_pred)
    #find the avg of squared_diff
    mae = np.mean(absolute_diff)
    return mae
```

Coefficient Of Determination (R^2)

It is a statistical measure used to evaluate the goodness of fit of a regression model. It represents the proportion of the variance in the dependent variable that is explained by the independent variables in the model. It is more useful when we have a linear relationship between dependent and independent variables.

R-squared ranges from 0 to 1, with 1 indicating a perfect fit and 0 indicating no fit at all.

Mathematically, it can be expressed as:

$$R\text{-squared} = 1 - \frac{SSR}{SST} = 1.0 - \frac{\sum_i^N (y_i - \hat{y}_i)^2}{\sum_i^N (y_i - \mu)^2}$$

Where :

- *SSR(Sum of Squared Residuals)* represents the sum of the squared differences between the predicted values and the actual values of the dependent variable.
- *SST(Sum of Squared Total)* represents the sum of the squared differences between the actual values of the dependent variable and the mean of the dependent variable.

```
"""
    Calculate R-squared value given predicted value and actual label

    Parameters:
        -- y_true (numpy array or list): True values
        -- y_pred (numpy array or list): Predicted values

    Returns:
        -- r^2 (float): Coefficient of determination(R^2)
    """

def r2_score(y_true, y_pred):

    #convert then into numpy array if not already
    y_true = np.array(y_true)
    y_pred = np.array(y_pred)
    #sum of squared residuals
    SSR = np.sum((y_true - y_pred)**2)
    #sum of squared total
    y_avg = np.mean(y_true)
    SST = np.sum((y_avg - y_true)**2)
    return 1 - (float(SSR)/float(SST))
```

Note : R-squared value can be negative due to outliers i.e when $MSE(model) > MSE(Baseline)$

Bounds of R-squared scores

[0, 1], with 0 the worst and 1 the best.

Weaknesses of R-squared scores

For comparative purposes, it's nice that R^2 is scaled between [0, 1]; as noted above, this lack of scaling makes mean squared error hard to interpret. But this also represents a trade-off: R^2 doesn't tell us about the magnitude of the errors.

Limitations:

- **Lack of information on prediction accuracy:** R-squared does not provide information about the accuracy of the model in making predictions. A high R-squared value does not necessarily guarantee accurate predictions, as the model may still have residual errors or may be overfitting the data.
- **Sensitivity to outliers:** R-squared can be affected by outliers in the data. Outliers can disproportionately influence the sum of squared residuals (SSR) component of R-squared, leading to an inflated or deflated R-squared value.
- **Inability to determine causality:** R-squared does not provide information about causality. Even if a regression model has a high R-squared value, it does not necessarily mean that the independent variables are causing the observed variation in the dependent variable. There may be other confounding variables or omitted variables that are driving the relationship.
- **Limited to linear relationships:** R-squared is a measure of the goodness of fit of a linear regression model, which assumes a linear relationship between the dependent and independent variables. If the relationship is non-linear, R-squared may not accurately reflect the goodness of fit.